

MODULE 0

DAY 1 • BEFORE MODULE 1

Foundations: How Your AI Coding Assistant Actually Works

A shared foundation before spec-driven development: connecting the Copilot skills you already use every day to what's actually happening underneath them.

What You'll Learn

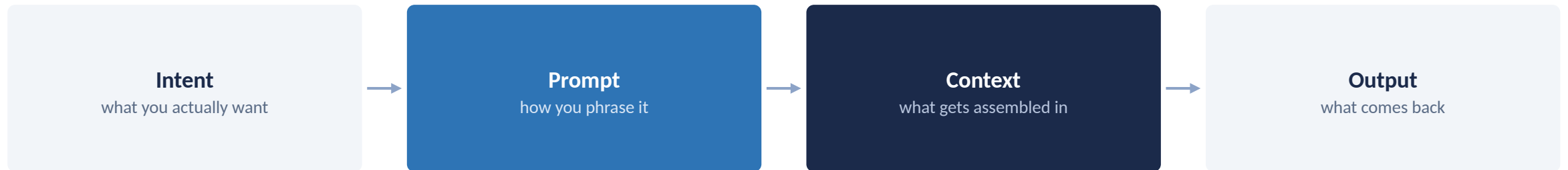
- 1 Trace intent to output, and meet the context window
- 2 Use .instructions.md well, including at monorepo scale
- 3 Explain how Copilot decides to call a tool
- 4 Place prompts, templates, agents, and skills on one map
- 5 Manage the context window deliberately, and hand it off

This isn't new material, it's naming what you already do.



Every module after this one assumes this shared vocabulary, worth 25 minutes now. Teams with years of daily Copilot use have hit this exact gap: fluent with the tool, unfamiliar with the mechanics underneath it.

Your Intent Passes Through Three Filters Before You See Output



Weak output usually traces back to one of the first three boxes, not the last one

The Context Window Holds Everything Copilot Can See Right Now

Open files & repo content

what's visible in your editor

Instructions files

persistent, auto-injected rules

Chat history

this conversation so far

Your current prompt

the newest addition

All four share one bounded window, that's the budget we'll come back to in Topic 6

NEXT

The File Behind Every Good Answer

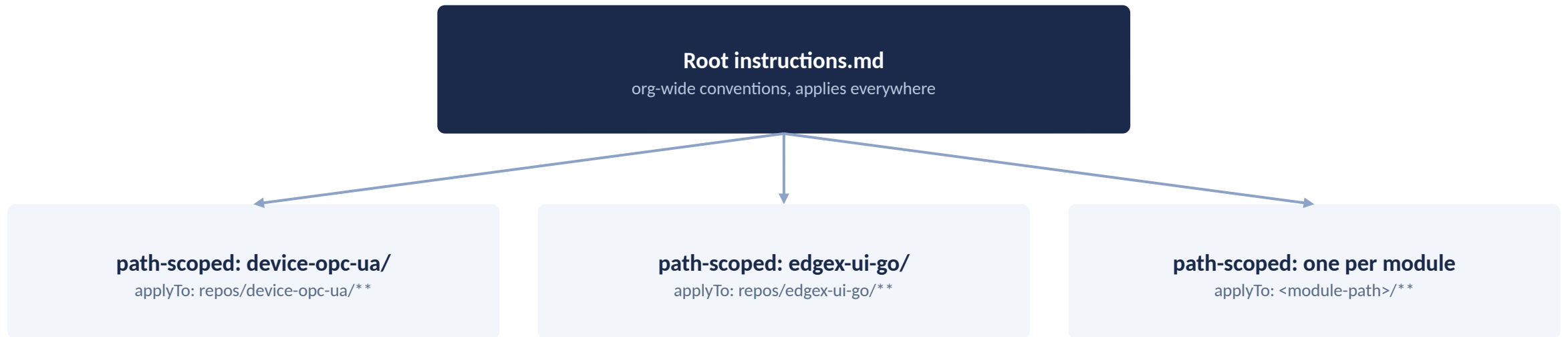
Why .instructions.md exists, and how to handle it once a project gets big

Without Instructions, Every Session Starts From Zero



That solves it for one repo. What about a hundred-module monorepo? Next.

One Root File, Many Scoped Files, at Monorepo Scale



Root file stays general; .github/instructions/.instructions.md with applyTo scopes the rest*

Real research says: modest gains, real costs, no silver bullet.



Gloaguen, Mündler, Müller,
Raychev, Vechev • ETH Zurich, 2026
• [arXiv:2602.11988](https://arxiv.org/abs/2602.11988)

The payoff is fewer rework cycles and style mismatches, not a guaranteed accuracy jump

CHECKPOINT

Let's Pause and Verify

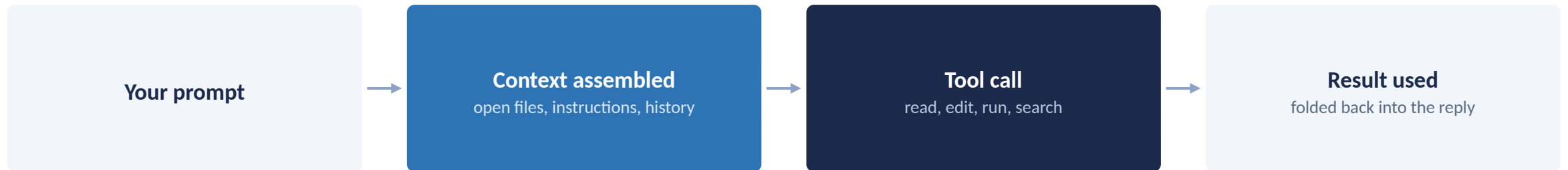
- 1 In your own words, what's actually inside the context window on a normal request?
- 2 If our monorepo has 80 modules, should we write one instructions.md or several, and why?

NEXT

What Copilot Actually Does With All of This

Tool calls, and what actually goes inside the instructions file

Copilot Decides When to Call a Tool, You Shape What It Sees



That assembly step is exactly what instructions.md controls, which is why Topic 2 came first

Five Sections Turn a Blank Page Into a Working File

1. Project overview

the elevator pitch: what, who, why

2. Tech stack

languages, frameworks, key libraries

3. Coding guidelines

style, testing, security expectations

4. Project structure

where things live, and why

5. Resources & tools

scripts, MCP servers, automation

An imperfect file with these five sections beats a perfect file that never gets written

CHECKPOINT

Let's Pause and Verify

1 What's one tool call Copilot might make on a typical request in our repo?

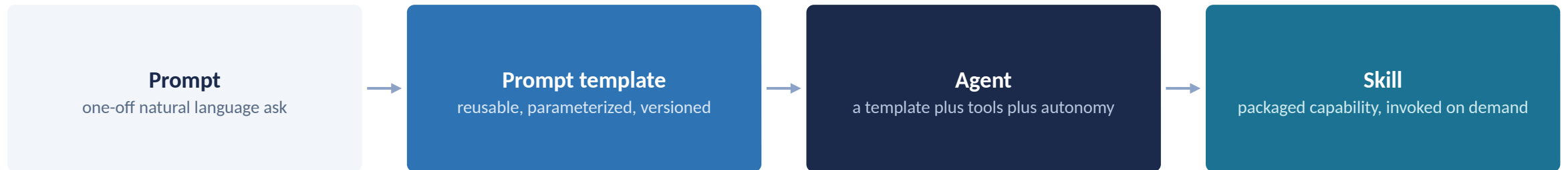
2 Name one of the five sections an instructions.md file should cover.

NEXT

The Bigger Picture

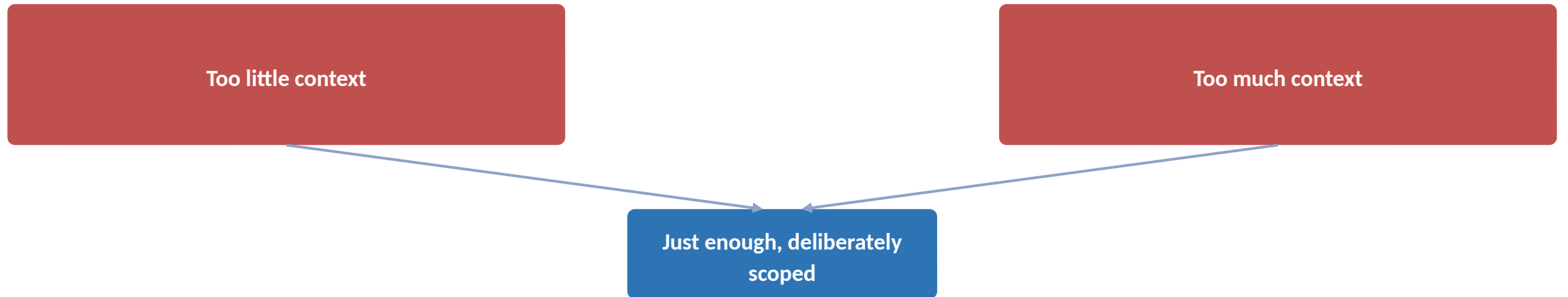
Where prompts, agents, and skills fit, and how to manage the window deliberately

Everything You Already Use Sits Somewhere on This Ladder



Every layer draws from the same limited resource: the context window from Topic 1

More context isn't free, and less context isn't safe.



Here's what that budget actually costs, in real and estimated numbers, next

Instructions files cost more upfront, discipline wins it back.

Measured • ETH Zurich, 2026

instruction files raise inference cost ~19% to 23% on average, mainly from more exploration & testing



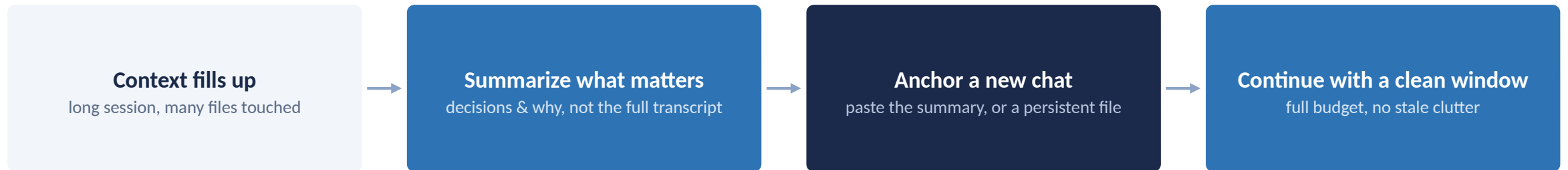
Illustrative estimate, not measured here

output tokens are commonly priced several times higher than input; caching and explicit brevity rules can meaningfully cut spend, exact % varies by provider and workload

*budget for the measured cost,
validate the estimated savings on
your own repo*

Right-hand numbers are planning assumptions to test, not a published result, treat them as a hypothesis

Summarize, Anchor, Continue: A Repeatable Handoff



The same discipline .instructions.md gives a repo, applied to a single conversation

Four Habits That Make Instructions Files Earn Their Cost

Write human-curated, specific rules

name the exact stack & versions, skip generic phrasing like "write clean code"

Command explicit brevity

tell it to skip filler, protects the output-token budget (Topic 6)

Layer root + path-scoped files

org-wide rules at root, module rules via applyTo globs (Topic 2)

Commit to .github/, not personal setup

every developer inherits the same guardrails automatically

This is Topic 2 and Topic 4, turned into four things to try this week

Key Takeaways

- 1 Intent becomes prompt, context, then output, weak output usually traces back to the first three, not the model.
- 2 Instructions.md pays context costs once instead of every prompt; at monorepo scale, pair a root file with path-scoped ones.
- 3 The real research shows modest, not-always-significant gains, the payoff is consistency and fewer rework cycles, not a guaranteed accuracy jump.
- 4 Prompts, templates, agents, and skills are the execution layer: the ladder you'll see structured formally as SDD in Module 1.
- 5 Next: Module 1, From Vibe Coding to Spec-Driven Development, starts now, building directly on everything in this module.